

MANUEL QUICK REF

VISUAL BASIC .NET

Formation 3 jours - 21 heures - version orientée live coding

J1	Algorithmique, Visual Studio, variables, types, conversions, opérateurs, lisibilité
J2	Conditions, boucles, tableaux, Sub, Function, paramètres, ByVal / ByRef
J3	Objets, SQLite / ADO.NET, WinForms, débogage, tests unitaires MSTest
Fil rouge	Mini-application de gestion de contacts
Usage	Révision rapide, aide pendant les TP, syntaxe essentielle, diagnostic

Principe - Ce manuel ne remplace pas les supports enrichis. Il condense les notions réellement traitées pendant les trois journées pour retrouver rapidement une syntaxe, une règle de décision, un exemple ou une méthode de diagnostic.

0. Comment utiliser ce Quick Ref

Chercher d'abord le besoin, puis la syntaxe. Chaque fiche répond à une question pratique : comment lire une saisie ? quand utiliser If ou Select Case ? comment parcourir un tableau ? comment créer un Contact ? comment diagnostiquer un bug ?

Repère	À quoi il sert
SYNTAXE	Bloc minimal directement réutilisable.
RÈGLE	Décision ou principe à retenir.
PIÈGE	Erreur fréquente observée dans le cours.
DEBUG	Méthode de diagnostic ou observation Visual Studio.
FIL ROUGE	Application gestion de contacts utilisée sur les trois jours.

Carte globale du cours

Jour	Question principale	Compétences clés
J1	Comment transformer un besoin en premier programme ?	Pseudo-code, projet Console, variables, types, conversions, opérateurs, expressions booléennes, lisibilité.
J2	Comment faire décider, répéter et structurer le programme ?	If/ElseIf, Select Case, While/Do/For, tableaux, recherche, Sub, Function, paramètres, ByVal/ByRef.
J3	Comment structurer une petite application et la fiabiliser ?	Classe Contact, SQLite/ADO.NET, WinForms, requête paramétrée, debugger, MSTest.

Méthode de travail - Pendant les exercices : prédire -> coder -> exécuter -> observer -> diagnostiquer -> modifier une seule chose -> relancer.

J1 - Fondements et premiers programmes

Du besoin au premier programme : algorithmique, console, variables, types, conversions, opérateurs et lisibilité.

1. Algorithme, programme et chaîne d'exécution

Terme	Définition rapide
Algorithme	Suite ordonnée d'étapes décrivant une solution indépendamment du langage.
Programme	Traduction exécutable d'un traitement dans un langage.
Code source	Texte Visual Basic écrit par le développeur.
Compilation	Vérification et construction du programme avant exécution.
Exécution	Lancement du programme construit.

Modèle mental - Entrée -> Traitement -> Sortie. Cette structure réapparaît dans presque tous les exercices du cours.

Pseudo-code minimal

```
LIRE prix
LIRE quantite
total <- prix * quantite
AFFICHER total
```

Le pseudo-code sert à raisonner sur l'ordre des opérations avant de se concentrer sur la syntaxe Visual Basic.

2. Structure minimale d'une application Console

```
Option Strict On

Module Program
    Sub Main()
        Console.WriteLine("Bonjour Visual Basic !")
    End Sub
End Module
```

Élément	Rôle
Option Strict On	Rend les conversions implicites risquées plus visibles et encourage des conversions explicites.
Module Program	Conteneur du code du petit programme console.
Sub Main()	Point d'entrée du programme.
Console.WriteLine	Affiche une ligne dans la console.

PIÈGE - Compiler et exécuter ne sont pas la même chose : un code peut compiler puis produire un mauvais résultat à l'exécution.

3. Variables, types et constantes

```
Dim age As Integer = 25
Dim prix As Decimal = 19.99D
Dim nom As String = "Nora"
Dim actif As Boolean = True
Dim temperature As Double = 21.5
Dim dateInscription As Date = Date.Now
Const TauxTVA As Decimal = 0.2D
```

Type	Utilisation typique	Exemple
Integer	Nombre entier	age, quantite
Double	Nombre avec décimales	temperature
Decimal	Calcul décimal/monétaire	prix, total
String	Texte	nom, téléphone, email
Boolean	Vrai / faux	estValide, trouve
Date	Date et heure	dateInscription

RÈGLE - Une variable possède un nom, un type et une valeur. Choisir le type à partir du sens de la donnée, pas au hasard.

Nommage - Préférer prixUnitaire, nombreContacts ou estValide à x, y ou flag1. Le nom doit raconter le rôle de la donnée.

4. Lire et afficher avec la console

```
Console.Write("Nom : ")
Dim nom As String = Console.ReadLine()
Console.WriteLine("Bonjour " & nom)
```

Instruction	Effet
Console.Write	Affiche sans retour à la ligne.
Console.WriteLine	Affiche puis passe à la ligne suivante.
Console.ReadLine	Lit une saisie utilisateur sous forme de String.
&	Concatène du texte en Visual Basic.

RÈGLE - ReadLine retourne du texte. Pour obtenir un nombre, il faut convertir la chaîne.

5. Conversions : Parse et TryParse

Conversion directe avec Parse

```
Dim age As Integer = Integer.Parse(Console.ReadLine())
```

Parse convient lorsqu'on suppose la valeur valide. Une saisie comme "abc" provoque une erreur de format à l'exécution.

Conversion contrôlée avec TryParse

```
Dim age As Integer
Dim ok As Boolean = Integer.TryParse(Console.ReadLine(), age)

Console.WriteLine("Conversion réussie : " & ok)
Console.WriteLine("Âge : " & age)
```

Choix	À retenir
Parse	Simple, mais échoue si le texte ne représente pas le type attendu.
TryParse	Retourne un Boolean indiquant si la conversion a réussi et place la valeur dans la variable fournie.

PIÈGE - Ne pas confondre une erreur de saisie avec un problème de syntaxe. Avec TryParse, le programme peut tester la réussite sans provoquer une exception de format.

6. Opérateurs essentiels

Famille	Opérateurs	Exemple / résultat
Arithmétique	+ - * /	10 + 5 -> 15
Division entière	\	5 \ 2 -> 2
Reste	Mod	5 Mod 2 -> 1
Puissance	^	2 ^ 3 -> 8
Comparaison	= <> < > <= >=	age >= 18 -> Boolean
Logique	And Or Not	age >= 18 And hasCard

RÈGLE - Une expression arithmétique produit une valeur numérique. Une comparaison produit un Boolean. And, Or et Not combinent des Boolean.

```
Dim age As Integer = 20
Dim hasCard As Boolean = True
Dim allowed As Boolean = age >= 18 And hasCard
Console.WriteLine(allowed)
```

7. Lisibilité et refactorisation J1

Fragile	Mieux
Dim x As Decimal = 19.99D	Dim prixUnitaire As Decimal = 19.99D
Dim y As Integer = 3	Dim quantite As Integer = 3
Dim z = x * y	Dim total As Decimal = prixUnitaire * quantite
Commentaire évident	Commentaire qui explique une intention non évidente

- Une indentation cohérente rend les blocs visibles.
- Un bon nom réduit le besoin de commentaires.
- Une modification de structure doit préserver le comportement attendu.
- Exécuter après chaque petite amélioration.

8. J1 - fiche syntaxe express

```
Dim age As Integer = 25
Dim prix As Decimal = 19.99D
Const TauxTVA As Decimal = 0.2D

Console.WriteLine("Nom : ")
Dim nom As String = Console.ReadLine()

Dim age2 As Integer
Dim ok As Boolean = Integer.TryParse(Console.ReadLine(), age2)

Dim total As Decimal = prix * 3
Dim majeur As Boolean = age >= 18
Dim autorise As Boolean = majeur And ok
```

Checklist J1 - Le programme compile ; il a été exécuté avec au moins deux jeux de données ; les noms sont explicites ; chaque conversion est comprise ; le résultat d'une expression peut être prédit avant Run.

J2 - Contrôle du programme et modularisation

Décider, répéter, stocker plusieurs valeurs et découper le programme en traitements réutilisables.

9. Conditions : If, ElseIf, Else

```
If age >= 18 Then
    Console.WriteLine("Majeur")
Else
    Console.WriteLine("Mineur")
End If
```

Plusieurs cas ordonnés

```
If score >= 90 Then
    Console.WriteLine("Excellent")
ElseIf score >= 70 Then
    Console.WriteLine("Bien")
ElseIf score >= 50 Then
    Console.WriteLine("Passable")
Else
    Console.WriteLine("Insuffisant")
End If
```

PIÈGE - Dans une chaîne ElseIf, la première condition vraie gagne. L'ordre des seuils fait donc partie de la logique.

Besoin	Structure naturelle
Exécuter quelque chose seulement si une règle est vraie	If...Then
Choisir entre deux cas exclusifs	If...Then...Else
Gérer des seuils ou plusieurs intervalles	If...Elseif...Else
Choisir parmi plusieurs valeurs discrètes d'une même variable	Select Case

10. Select Case : menus et valeurs discrètes

```
Select Case choice
    Case "1"
        Console.WriteLine("Ajouter")
    Case "2"
        Console.WriteLine("Afficher")
    Case "3"
        Console.WriteLine("Rechercher")
    Case "0"
        Console.WriteLine("Au revoir")
    Case Else
        Console.WriteLine("Choix invalide")
End Select
```

RÈGLE - Select Case est particulièrement lisible quand on compare une même variable à plusieurs valeurs distinctes, par exemple les options d'un menu.

11. Boucles : choisir la bonne forme

Boucle	Quand l'utiliser	Point de vigilance
While	Répéter tant qu'une condition est vraie	La condition doit pouvoir évoluer.
Do While	Même logique, syntaxe centrée sur "tant que"	Vérifier le moment du test.
Do Until	Répéter jusqu'à ce qu'une condition devienne vraie	Formuler clairement la condition d'arrêt.
For...Next	Nombre de répétitions connu	Bien lire première et dernière valeur.
For Each	Parcourir chaque élément d'une collection / tableau	On manipule directement l'élément.

While avec compteur

```
Dim i As Integer = 1
While i <= 5
    Console.WriteLine(i)
    i += 1
End While
```

PIÈGE - Si la variable de contrôle ne change jamais, la condition peut rester vraie indéfiniment : boucle infinie.

Do Until pour un menu

```
Do Until choice = "0"
    Console.WriteLine("Choix (0 pour quitter) : ")
    choice = Console.ReadLine()
Loop
```

For...Next

```
For i As Integer = 1 To 10
    Console.WriteLine(n & " x " & i & " = " & (n * i))
Next
```

12. Compteur et accumulateur

```
Dim total As Integer = 0

For i As Integer = 1 To 5
    Console.WriteLine("Valeur " & i & " : ")
    Dim value As Integer = Integer.Parse(Console.ReadLine())
    total += value
Next

Console.WriteLine("Total = " & total)
```

RÈGLE - Un accumulateur s'initialise avant la boucle puis se met à jour à chaque tour. S'il est recréé dans la boucle, la valeur précédente est perdue.

13. Tableaux à une dimension

```
Dim names(9) As String
Dim count As Integer = 0

names(count) = "Alice"
count += 1

For i As Integer = 0 To count - 1
    Console.WriteLine(names(i))
Next
```

Notion	Règle rapide
Length	Nombre total de cases du tableau.
Dernier index	Length - 1.
count	Nombre d'éléments réellement utilisés dans un tableau partiellement rempli.
Index	Position d'un élément, généralement à partir de 0.

PIÈGE - Pour un tableau de Length 5, les index valides vont de 0 à 4. Length n'est pas un index valide.

14. Recherche dans un tableau

```
Dim found As Boolean = False

For i As Integer = 0 To count - 1
    If names(i).ToLower() = target.ToLower() Then
        Console.WriteLine(names(i))
        found = True
    End If
Next

If Not found Then
    Console.WriteLine("Contact non trouvé.")
End If
```

RÈGLE - Avant la recherche, found démarre à False parce qu'aucun élément n'a encore été trouvé. La valeur devient True uniquement lorsqu'un match est observé.

15. Matrice 2D : idée minimale

```
Dim grid(1, 2) As Integer

grid(0, 0) = 10
grid(0, 1) = 20
grid(0, 2) = 30
grid(1, 0) = 40
grid(1, 1) = 50
grid(1, 2) = 60
```

Une matrice peut être parcourue avec deux boucles imbriquées : une pour les lignes, une pour les colonnes. Le cours se limite à comprendre cette organisation.

16. Sub : extraire une action

```
Sub ShowMenu()
    Console.WriteLine("1 - Ajouter")
    Console.WriteLine("2 - Afficher")
    Console.WriteLine("3 - Rechercher")
    Console.WriteLine("0 - Quitter")
End Sub
```

Sub avec paramètres

```
Sub PrintContact(name As String, phone As String)
    Console.WriteLine(name & " - " & phone)
End Sub
```

RÈGLE - Une Sub réalise principalement une action et ne fournit pas de valeur de retour à l'appelant.

17. Function : calculer et retourner une valeur

```
Function Add(a As Integer, b As Integer) As Integer
    Return a + b
End Function

Dim result As Integer = Add(4, 7)
```

Function métier lisible

```
Function IsValidName(name As String) As Boolean
    Return name.Trim().Length >= 2
End Function
```

Sub	Function
Action principale	Produit une valeur
Pas de valeur de retour	Type de retour déclaré avec As ...
ShowMenu()	result = Add(a, b)
PrintContact(...)	IsValidName(name)

18. Paramètres, arguments et portée locale

```
Function Multiply(a As Integer, b As Integer) As Integer
    Return a * b
End Function

Dim total As Integer = Multiply(3, 5)
```

Terme	Dans l'exemple
Paramètres	a et b dans la définition.
Arguments	3 et 5 au moment de l'appel.
Valeur de retour	15.
Variable réceptrice	total.

Portée - Une variable locale est accessible uniquement dans le bloc où elle est déclarée. Cela limite les modifications accidentelles.

19. ByVal et ByRef

ByVal : réaffectation locale

```
Sub Increase(ByVal number As Integer)
    number += 1
End Sub
```

```
Dim x As Integer = 10
Increase(x)
Console.WriteLine(x) ' 10
```

ByRef : modification visible chez l'appelant

```
Sub IncreaseByRef(ByRef number As Integer)
    number += 1
End Sub
```

```
Dim x As Integer = 10
IncreaseByRef(x)
Console.WriteLine(x) ' 11
```

RÈGLE - Commencer par ByVal. Utiliser ByRef uniquement lorsque la modification de la variable de l'appelant fait explicitement partie du contrat de la procédure.

20. J2 - règles de décision express

Question	Réponse rapide
Une règle vrai/faux ?	If
Des seuils ?	Elseif ordonné
Un menu 1/2/3/0 ?	Select Case
Répéter tant qu'une condition est vraie ?	While / Do While
Répéter jusqu'à une condition ?	Do Until
Répéter N fois ?	For...Next
Parcourir un tableau ?	For ou For Each
Action réutilisable ?	Sub
Valeur calculée réutilisable ?	Function
Modifier volontairement la variable appelante ?	ByRef, sinon ByVal

Transition J3 - Les tableaux parallèles names() et phones() fonctionnent, mais ils séparent les données d'un même contact. J3 introduit une classe Contact pour regrouper ces données.

J3 - Objets, données, interface, débogage et tests

Regrouper les données, persister en base, relier WinForms, diagnostiquer méthodiquement et automatiser une vérification.

21. Classe, objet, propriété, méthode, constructeur

Notion	Définition rapide
Classe	Type créé par le développeur pour représenter une chose et ses comportements.
Objet / instance	Valeur concrète créée à partir d'une classe.
Propriété	Donnée accessible sur un objet.
Méthode	Traitement appartenant à un type ou à un objet.
Constructeur	Procédure exécutée lors de New.
Encapsulation	Regrouper et protéger l'état et les règles d'un objet.

```
Public Class Contact
    Public Property Name As String
    Public Property Phone As String
    Public Property Email As String

    Public Sub New(name As String, phone As String, email As String)
        Me.Name = name
        Me.Phone = phone
        Me.Email = email
    End Sub

    Public Function DisplayLabel() As String
        Return Name & " - " & Phone & " - " & Email
    End Function
End Class
```

Créer une instance

```
Dim alice As New Contact("Alice", "0600000001", "alice@example.com")
Console.WriteLine(alice.DisplayLabel())
```

RÈGLE - La classe décrit le modèle ; New crée une instance concrète. Plusieurs objets d'une même classe possèdent chacun leurs propres valeurs.

22. Plusieurs objets et Nothing

```
Dim contacts() As Contact = {
    New Contact("Alice", "0600000001", "alice@example.com"),
    New Contact("Bob", "0600000002", "bob@example.com")
}

For Each contact As Contact In contacts
    Console.WriteLine(contact.DisplayLabel())
Next
```

Tester Nothing avant utilisation

```
Dim result As Contact = FindByName(contacts, "Bob")

If result Is Nothing Then
    Console.WriteLine("Aucun contact trouvé.")
Else
    Console.WriteLine(result.DisplayLabel())
End If
```

PIÈGE - Accéder à result.Name alors que result vaut Nothing déclenche une NullReferenceException à l'exécution.

23. Organisation simple des responsabilités

Zone	Responsabilité
Interaction	Console ou WinForms : lire la saisie, afficher un résultat.
Logique	Validation, recherche, règles simples.
Données	Tableau puis base SQLite.
Contact	Objet représentant les données et comportements d'un contact.

Refactorisation - Améliorer la structure sans changer le comportement attendu. Après chaque extraction ou déplacement : compiler, relancer le même scénario, comparer le résultat.

24. SQLite : vocabulaire minimal

Terme	Dans le fil rouge Contacts
Table	Contacts
Ligne	Un contact
Colonne	Name, Phone, Email
Clé primaire	Id
Requête	SELECT ... WHERE ...
Paramètre	@search

```
CREATE TABLE Contacts (
    Id INTEGER PRIMARY KEY AUTOINCREMENT,
    Name TEXT NOT NULL,
    Phone TEXT NOT NULL,
    Email TEXT NOT NULL
);
```

25. ADO.NET + SQLite : trajet conceptuel

Trajet - Application -> Connection -> Command -> Database -> Reader -> Contact -> Affichage.

Objet	Question à laquelle il répond
Connection	À quelle base voulons-nous parler ?
Command	Quelle requête voulons-nous exécuter ?
Parameter	Quelle valeur utilisateur est fournie séparément du SQL ?
Reader	Comment lire les lignes renvoyées ?

Objet	Question à laquelle il répond
Contact	Comment représenter chaque ligne côté application ?

Package utilisé dans le cours

```
dotnet add package Microsoft.Data.Sqlite
```

Connexion minimale

```
Imports Microsoft.Data.Sqlite

Dim connectionString As String = "Data Source=contacts.db"

Using connection As New SqliteConnection(connectionString)
    connection.Open()
    Console.WriteLine("Connexion ouverte.")
End Using
```

RÈGLE - Using libère automatiquement la ressource à la fin du bloc. Ouvrir le plus tard possible et libérer le plus tôt possible.

26. Requête paramétrée

```
Dim sql As String =
    "SELECT Name, Phone, Email FROM Contacts " &
    "WHERE Name LIKE @search ORDER BY Name"

Using command As New SqliteCommand(sql, connection)
    command.Parameters.AddWithValue("@search", "%" & search & "%")

    Using reader As SqliteDataReader = command.ExecuteReader()
        While reader.Read()
            Dim contact As New Contact(
                reader.GetString(0),
                reader.GetString(1),
                reader.GetString(2)
            )
        End While
    End Using
End Using
```

RÈGLE - Les valeurs utilisateur sont transmises séparément du texte SQL. Ne pas construire la requête en concaténant directement la saisie utilisateur.

27. FindContacts : forme de référence

```
Public Function FindContacts(search As String) As List(Of Contact)
    Dim results As New List(Of Contact)()

    Using connection As New SqlConnection(ConnectionString)
        connection.Open()

        Dim sql As String =
            "SELECT Name, Phone, Email FROM Contacts " &
            "WHERE Name LIKE @search ORDER BY Name"

        Using command As New SqlCommand(sql, connection)
            command.Parameters.AddWithValue("@search", "%" & search & "%")

            Using reader As SqlDataReader = command.ExecuteReader()
                While reader.Read()
                    results.Add(New Contact(
                        reader.GetString(0),
                        reader.GetString(1),
                        reader.GetString(2)
                    ))
                End While
            End Using
        End Using
    End Using

    Return results
End Function
```

List(Of Contact) est utilisée ici comme collection redimensionnable pratique pour retourner zéro, un ou plusieurs contacts.

28. WinForms : même logique, autre interaction

Console	WinForms
Console.ReadLine	TextBox.Text
Commande saisie / menu	Button.Click
Console.WriteLine	ListBox.Items / DataGridView
Boucle explicite d'interaction	Application en attente d'événements

Gestionnaire Click de référence

```
Private Sub btnSearch_Click(sender As Object, e As EventArgs) _
    Handles btnSearch.Click

    Dim search As String = txtSearch.Text.Trim()

    If String.IsNullOrEmpty(search) Then
        MessageBox.Show("Saisissez un nom à rechercher.")
        txtSearch.Focus()
        Return
    End If

    Dim contacts As List(Of Contact) = FindContacts(search)
    lstResults.Items.Clear()

    If contacts.Count = 0 Then
        lstResults.Items.Add("Aucun résultat.")
        Return
    End If

    For Each contact As Contact In contacts
        lstResults.Items.Add(contact.DisplayLabel())
    Next
End Sub
```

RÈGLE - WinForms ne remplace pas la logique métier : il change surtout la manière de recevoir une entrée et d'afficher une sortie.

29. Classer les erreurs avant de corriger

Type	Symptôme typique	Exemple
Compilation	Le projet ne se construit pas	Nom de propriété incorrect, syntaxe invalide.
Exécution	Le programme démarre puis échoue	Nothing utilisé, chemin de base incorrect.
Logique	Le programme tourne mais le résultat est faux	Paramètre LIKE sans % ou critère modifié.

DEBUG - Ne pas modifier le code au hasard. Décrire d'abord le symptôme et le résultat attendu, puis formuler une hypothèse vérifiable.

30. Méthode générale de diagnostic en 6 questions

- Quel est le symptôme observable ?
- À quel moment apparaît-il : compilation, démarrage, clic, requête, affichage ?
- Quel était le résultat attendu ?
- Quelle est la dernière étape dont on sait qu'elle fonctionne ?
- Quelle valeur ou condition peut expliquer l'écart ?
- Comment vérifier cette hypothèse par une observation plutôt que par une supposition ?

RÈGLE - Le débogage n'est pas "modifier jusqu'à ce que ça marche". C'est formuler une hypothèse puis la vérifier.

31. Débogueur Visual Studio : outils essentiels

Outil	Usage rapide
Breakpoint	Suspend volontairement l'exécution sur une ligne.
Locals	Montre les variables disponibles dans le contexte courant.
Watch	Suit une variable ou une expression choisie.
Step Over	Exécute la ligne sans entrer dans la fonction appelée.
Step Into	Entre dans la fonction appelée.
Step Out	Termine la fonction courante et revient à l'appelant.
Call Stack	Montre la chaîne d'appels active.

Checklist recherche SQL - Vérifier search ; vérifier FindContacts ; inspecter la chaîne de connexion ; inspecter @search ; vérifier reader.Read ; vérifier les objets Contact ; vérifier la collection retournée ; vérifier l'affichage WinForms.

32. Tests unitaires MSTest

Un test unitaire automatise une vérification jouable sur un petit comportement isolé. Le cours utilise Arrange - Act - Assert (AAA).

Phase	Question	Exemple
Arrange	Quelles données prépare-t-on ?	email = "alice@example.com"
Act	Quelle action teste-t-on ?	IsValidEmail(email)
Assert	Quel résultat attend-on ?	True

```
Public Function IsValidEmail(email As String) As Boolean
    Return Not String.IsNullOrEmpty(email) AndAlso
        email.Contains("@")
End Function
```

```
Imports Microsoft.VisualStudio.TestTools.UnitTesting
```

```
<TestClass>
Public Class ContactTests

    <TestMethod>
    Public Sub IsValidEmail_WithAt_ReturnsTrue()
        ' Arrange
        Dim email As String = "alice@example.com"

        ' Act
        Dim result As Boolean = IsValidEmail(email)

        ' Assert
        Assert.IsTrue(result)
    End Sub
End Class
```

État	Interprétation
Vert	Pour ce scénario, le résultat observé correspond à l'attendu.
Rouge	Le comportement observé ne correspond pas à l'attendu ; lire le message avant de corriger.

Régression - Un test aide à détecter qu'un comportement qui fonctionnait auparavant a été cassé par une modification.

SYNTHÈSE - Fil rouge Contacts - trajet complet

Relier les trois journées dans une seule carte mentale.

33. Évolution du fil rouge sur les 3 jours

Jour	Évolution
J1	Saisir et afficher un contact en console ; manipuler variables, conversions, calculs et expressions booléennes.
J2	Stocker plusieurs contacts, menu répétitif, ajout / affichage / recherche, extraction en Sub et Function.
J3	Créer la classe Contact, connecter SQLite, rechercher via ADO.NET, afficher dans WinForms, diagnostiquer et tester.

34. Trajet complet d'une recherche

Étape	Ce qui se passe
1. Interface	L'utilisateur saisit un texte dans txtSearch et clique sur btnSearch.
2. Validation	Le texte est Trim puis contrôlé.
3. Appel	Le Click appelle FindContacts(search).
4. Connexion	ADO.NET ouvre SQLite via SqlConnection.
5. Commande	SqlCommand contient le SQL et le paramètre @search.
6. Lecture	SqlDataReader lit chaque ligne renvoyée.
7. Objets	Chaque ligne devient un objet Contact.
8. Retour	FindContacts retourne une List(Of Contact).
9. Affichage	La ListBox ou le DataGridView présente les résultats.

Point de contrôle final - Savoir raconter ce trajet sans regarder le code prouve que les notions ne sont plus isolées.

35. Tableau de décision global

Besoin	Outil / syntaxe
Stocker une donnée simple	Variable Dim ... As ...
Valeur qui ne doit pas changer	Const
Lire du texte	Console.ReadLine
Convertir une saisie avec contrôle	TryParse
Décider	If / ElseIf / Else
Menu à valeurs discrètes	Select Case
Répéter tant qu'une règle reste vraie	While / Do While
Répéter jusqu'à un état	Do Until
Répéter N fois	For...Next
Stocker plusieurs valeurs du même type	Tableau
Réaliser une action réutilisable	Sub
Calculer / retourner une valeur	Function + Return
Regrouper les données d'un contact	Class Contact
Créer un contact concret	New Contact(...)

Besoin	Outil / syntaxe
Conserver les données entre exécutions	SQLite
Envoyer une valeur utilisateur au SQL	Paramètre @search
Réagir à un clic	Handles btnSearch.Click
Observer l'état réel à l'exécution	Breakpoint / Locals / Watch
Vérifier automatiquement une règle	MSTest + AAA

36. Erreurs fréquentes - anti-sèche

Erreur	Symptôme	Réflexe
Parse sur "abc"	Exception de format	Préférer TryParse si saisie utilisateur.
ElseIf mal ordonné	Mauvaise catégorie	Revoir la première condition vraie.
While sans mise à jour	Boucle infinie	Vérifier ce qui rapproche de l'arrêt.
For jusqu'à Length	Index hors limites	Dernier index = Length - 1.
total initialisé dans la boucle	Cumul perdu	Initialiser avant la boucle.
found = True au départ	Recherche toujours positive	Démarrer à False.
Tout laisser dans Main	Code long / difficile à lire	Extraire Sub / Function.
ByRef par habitude	Effets de bord	Utiliser seulement si modification voulue.
Oublier New	Référence Nothing	Créer l'instance.
Utiliser Nothing	NullReferenceException	Tester Is Nothing.
Concaténer saisie dans SQL	Fragilité / sécurité	Utiliser une requête paramétrée.
Mauvais chemin contacts.db	Erreur accès / table absente	Inspecter la chaîne de connexion.
Click non relié	Bouton sans effet	Vérifier Handles btnSearch.Click.
Liste non vidée	Résultats qui s'accumulent	Items.Clear avant ajout.
LIKE sans %	0 résultat partiel	Inspecter @search dans Watch.

37. Checklist avant de dire "ça marche"

- Le projet compile sans erreur.
- Le programme a été exécuté avec plusieurs jeux de données.
- Un cas limite ou invalide a été essayé lorsque c'est pertinent.
- Les noms de variables, Sub, Function et propriétés expriment leur intention.
- Chaque boucle possède une condition d'arrêt compréhensible.
- Les index de tableau restent dans les limites.
- Les fonctions renvoient une valeur cohérente avec leur type de retour.
- ByRef n'est utilisé que si la modification de l'appelant est volontaire.
- Les objets peuvent être Nothing : le cas est traité avant accès si nécessaire.
- Les valeurs SQL sont paramétrées.
- Un bug est diagnostiqué à partir d'un symptôme et d'une hypothèse, pas au hasard.
- Un test rouge est lu avant correction ; le scénario est rejoué après correction.

38. Glossaire ultra-rapide

Terme	Définition
Boolean	Valeur True ou False.
Expression	Combinaison de valeurs / variables / opérateurs produisant une valeur.
Paramètre	Variable déclarée dans la signature d'une Sub ou Function.
Argument	Valeur fournie au moment de l'appel.
Portée	Zone dans laquelle un nom est accessible.
Effet de bord	Modification observable autre que la valeur de retour.
Instance	Objet concret créé à partir d'une classe.
Nothing	Absence de référence vers un objet.

Terme	Définition
ADO.NET	API .NET d'accès aux sources de données.
Requête paramétrée	SQL dont les valeurs sont fournies séparément du texte de la requête.
Événement	Fait auquel l'interface réagit, par exemple un clic.
Breakpoint	Point où le debugger suspend volontairement l'exécution.
Régression	Comportement qui fonctionnait puis casse après une modification.

39. Syntaxe essentielle - une page de révision

```
' Variables / constantes
Dim age As Integer = 25
Const TauxTVA As Decimal = 0.2D

' Saisie / conversion
Dim raw As String = Console.ReadLine()
Dim value As Integer
Dim ok As Boolean = Integer.TryParse(raw, value)

' Condition
If ok Then
    Console.WriteLine(value)
Else
    Console.WriteLine("Saisie invalide")
End If

' Boucle
For i As Integer = 0 To values.Length - 1
    Console.WriteLine(values(i))
Next

' Fonction
Function IsAdult(age As Integer) As Boolean
    Return age >= 18
End Function

' Objet
Dim contact As New Contact("Alice", "0600000001", "alice@example.com")

' Nothing
If contact Is Nothing Then
    Return
End If
```

40. Les 12 règles à retenir après les 3 jours

- 1. Partir du besoin avant de chercher la syntaxe.
- 2. Lire une saisie comme du texte puis convertir explicitement si nécessaire.
- 3. Une condition est une expression qui produit True ou False.
- 4. Une boucle doit avoir une raison claire de continuer et une manière claire de finir.
- 5. Le dernier index d'un tableau est Length - 1.
- 6. Une Sub réalise une action ; une Function produit une valeur.
- 7. Préférer ByVal ; utiliser ByRef seulement pour une modification volontaire de l'appelant.
- 8. Regrouper les données liées dans un objet quand les tableaux parallèles deviennent fragiles.
- 9. Séparer interaction, logique et données suffisamment pour comprendre le rôle de chaque partie.

- 10. Paramétrer le SQL au lieu de concaténer les saisies utilisateur.
- 11. Déboguer par hypothèse et observation, puis rejouer le scénario qui échouait.
- 12. Un test unitaire vérifie automatiquement un comportement précis et aide à prévenir les régressions.

Fin du Quick Ref - Si vous ne savez plus quelle syntaxe utiliser, revenez d'abord au tableau de décision global, puis au bloc de code minimal correspondant.